

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.



Autor: Carlos Moreno Martínez.
cmorenomartinez@educa.madrid.org



UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

1.- Introducción.

1.1.- El Modelo CRUD.



- Una Base de Datos debe permitir cuatro operaciones básicas sobre los datos que contiene. Dichos datos se conocen con el acrónimo **CRUD** (**C**reate, **R**ead, **U**ppdate, **D**elte).

En el capítulo anterior hemos visto como se manipulaban las tablas que contenia una Base de Datos. En este veremos como **manipular los datos de una tabla**.

- Estas operaciones pueden involucrar a todos los registros de las tablas o bian a los datos que cumplan determinados criterios que marque el usuario.
- El éxito de una operación de manipulación de datos **está sujeto a las características y restricciones impuestas a la hora de crear los campos de la tabla y la definición de las restricciones (constraints) impuestas a estos campos**.
- El usuario debe tener la posibilidad de **cancelar una determinada operación**.
Dicha cancelación no debe suponer un problema para que los datos almacenados se encuentren en un estado fiable (**ROLLBACK**).

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

1.- Introducción.

1.2.- Enumeración de los comandos DML.

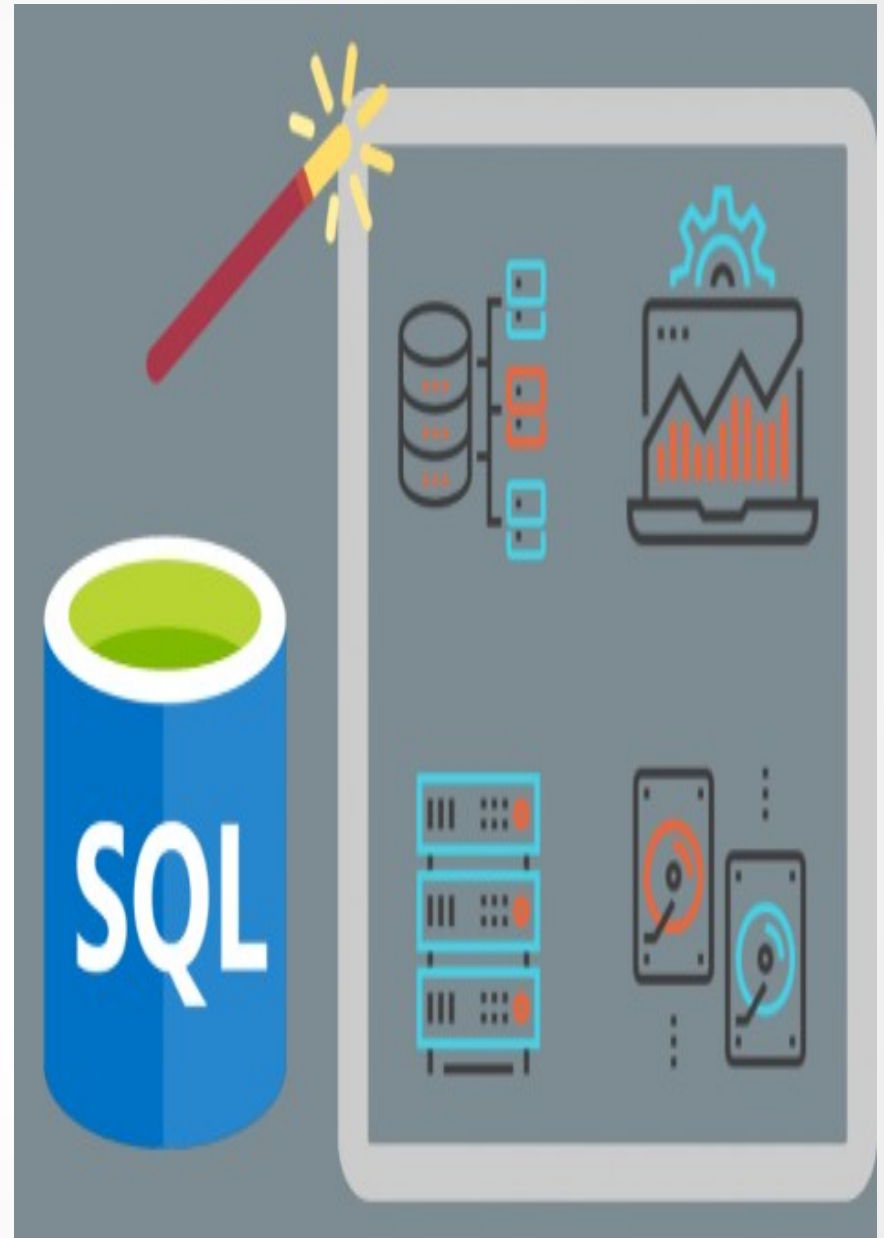
El lenguaje DML está formado por tres comandos de **manipulación de datos** y tres comandos de **gestión de transacciones**.

Los comandos de **manipulación de datos** son:

- **INSERT.** Añadir registros en las tablas.
- **UPDATE.** Modificar la información contenida.
- **DELETE.** Suprimir registros.

Los comandos para la **gestión de transacciones** son:

- **COMMIT.** Confirma los cambios realizados en las distintas tablas.
- **ROLLBACK.** Descarta los cambios realizados desde un determinado momento.
- **SAVEPOINT.** Crea un punto de retorno.



UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

1.- Introducción.

1.3.- Tabla de Ejemplo.

Tabla y datos de ejemplo.

Para los ejemplos utilizaremos la tabla EMPLEADOS que contiene una serie de registros. Las sentencias para generar la tabla y los datos son las siguientes:

```
CREATE TABLE EMPLEADOS (  
    EMPLEADOID NUMBER(4) PRIMARY KEY,  
    NOMBRE VARCHAR2(20),  
    PUESTO VARCHAR2(15),  
    SALARIO NUMBER(8,2)  
);  
  
INSERT INTO EMPLEADOS VALUES (1, 'GOMEZ, MARIA JOSE', 'TECNICO', 1850.25);  
  
INSERT INTO EMPLEADOS VALUES (2, 'JIMENEZ, JOSE LUIS', 'CONTABLE', 1752.25);  
  
INSERT INTO EMPLEADOS VALUES (3, 'CARBALLO, ALBERTO', 'TECNICO', 1712.56);  
  
INSERT INTO EMPLEADOS VALUES (4, 'SANTOS, ANA MARIA', 'ADMINISTRATIVO', 1378.95);  
  
INSERT INTO EMPLEADOS VALUES (5, 'PEREDA, GUSTAVO', 'COMERCIAL', 1492.78);  
  
COMMIT;
```


UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

2.- Inserción de registros en una tabla. La sentencia INSERT.

2.1.- Inserción básica de registros. Sentencia y características.

Sintaxis.

```
INSERT INTO TABLA  
    (COLUMNA1, COLUMNA2, ... , COLUMNAN)  
VALUES  
    (VALOR1, VALOR2, ... , VALORN) ;
```

- Los valores que se dan a las columnas deben coincidir con el tipo de dato definido en la columna.
- Los valores constantes de tipo carácter han de ir encerrados entre comillas simples (' '). Para los tipos de fecha conviene una usar la función **TO_DATE()**.
- No es obligatorio incluir todos los campos, en la lista tan solo los campos donde se incluyen valores. Los campos donde que no se incluyen en la lista se crearán con valor **NULL**.
- Es posible omitir la lista de columnas, si se indican todos los valores de cada campo y en el orden en el que se encuentran en la tabla.
- Es importante tener en cuenta que los valores para cada campo deben tener un formato con un valor adecuado (tipo de campo y que cumpla cual restricción).

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

2.- Inserción de registros en una tabla. La sentencia INSERT.

2.1.- Inserción básica de registros. Ejemplo.

```
INSERT INTO EMPLEADOS VALUES (6, 'TORRES, ALEJANDRO', 'DIRECTOR', 1893.64);
```

1 fila insertadas.

```
INSERT INTO EMPLEADOS (EMPLEADOID, NOMBRE, PUESTO)
VALUES
(7, 'RODAS, ELENA', 'COMERCIAL');
```

1 fila insertadas.

```
SELECT * FROM EMPLEADOS;
```

EMPLEADOID	NOMBRE	PUESTO	SALARIO
1	GOMEZ, MARIA JOSE	TECNICO	1850,25
2	JIMENEZ, JOSE LUIS	CONTABLE	1752,25
3	CARBALLO, ALBERTO	TECNICO	1712,56
4	SANTOS, ANA MARIA	ADMINISTRATIVO	1378,95
5	PEREDA, GUSTAVO	COMERCIAL	1492,78
6	TORRES, ALEJANDRO	DIRECTOR	1893,64
7	RODAS, ELENA	COMERCIAL	

7 filas seleccionadas.

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

2.- Inserción de registros en una tabla. La sentencia INSERT.

2.2.- Inserción múltiple de registros. INSERT ALL.

Sintaxis.

Es posible realizar la inserción no solo de varios registros en una misma tabla sino que múltiples registros en distintas tablas.

```
INSERT ALL
  INTO TABLA1 (COL1, COL2, ... , COLN) VALUES (VAL1.1, VAL1.2, ... ,VAL1.N)
  INTO TABLA2 (COL1, COL2, ... , COLN) VALUES (VAL2.1, VAL2.2, ... ,VAL2.N)
  .
  .
  INTO TABLAN (COL1, COL2, ... , COLN) VALUES (VALM.1, VALM.2, ... ,VALM.N)
SELECT * FROM DUAL;
```

- Puede ser una única tabla o bien varias tablas. **En cada línea INTO se define en que tabla se desea insertar el registro.**
- **Si una o varias inserciones no es posible realizarlas** (error de tipo de datos, sintaxis, violación de constraints, etc), se realiza un rollback de **TODAS** las operaciones anteriores y **NO SE REALIZAN** las que quedan pendientes. **Es decir, se anula la sentencia entera.**
- Por cada uno de los INTO se inserta un registro. En el caso de que inserten registros en varias tablas ligadas mediante integridad referencial, **se deben insertar los registros de forma que dicha integridad no se viole.**

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

2.- Inserción de registros en una tabla. La sentencia INSERT INTO.

2.2.- Inserción múltiples de registros. Ejemplo.

```
INSERT ALL
  INTO EMPLEADOS VALUES (8, 'SALAS, JORGE', 'TECNICO', 1456.24)
  INTO EMPLEADOS VALUES (9, 'GALLEGO, ARANCHA', 'TECNICO', 1678.54)
  INTO EMPLEADOS VALUES (10, 'RAMOS, SERGIO', 'ADMINISTRATIVO', 1468.98)
SELECT * FROM DUAL;
```

3 filas insertadas.

```
SELECT * FROM EMPLEADOS;
```

EMPLEADOID	NOMBRE	PUESTO	SALARIO
1	GOMEZ, MARIA JOSE	TECNICO	1850,25
2	JIMENEZ, JOSE LUIS	CONTABLE	1752,25
3	CARBALLO, ALBERTO	TECNICO	1712,56
4	SANTOS, ANA MARIA	ADMINISTRATIVO	1378,95
5	PEREDA, GUSTAVO	COMERCIAL	1492,78
6	TORRES, ALEJANDRO	DIRECTOR	1893,64
7	RODAS ELENA	COMERCIAL	
8	SALAS, JORGE	TECNICO	1456,24
9	GALLEGO, ARANCHA	TECNICO	1678,54
10	RAMOS, SERGIO	ADMINISTRATIVO	1468,98

10 filas seleccionadas.

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

2.- Inserción de registros en una tabla. La sentencia INSERT.

2.3.- Inserción de registros mediante una sentencia SQL.

Sintaxis.

```
INSERT INTO TABLA_INSERCIÓN (COL1, COL2, ... , COLN)  
[CONSULTA];
```

- Los datos que se insertan en los nuevos registros se rescatan mediante una consulta SQL a otra tabla.
- Se puede insertar hasta un **máximo de cien registros**.
- Los campos deben ser coherentes a nivel de tipo de dato y no violar la integridad referencial.

Ejemplo.

```
INSERT INTO GRANDESCUENTAS  
SELECT CLIENTEID, NOMBRE, FACTURACION  
FROM CLIENTES  
WHERE FACTURACION >= 100000;
```

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

3.- Actualización de registros. La sentencia UPDATE.

3.1.- Sentencia Básica. Sentencia y características.

Sintaxis.

```
UPDATE TABLA  
SET CAMPO1 = VALOR1 , CAMPO2=VALOR2, ... , CAMPON=VALORN  
WHERE [CONDICIÓN];
```

Los parámetros que aparecen los siguientes:

- **TABLA.** Nombre de la tabla en la que se desea modificar los registros. Solo es posible modificar una tabla de forma simultanea.
- **CAMPO=VALOR.** Se pueden especificar los nombres de campos que se deseen de la tabla indicada. Cada emparejamiento campo=valor debe separarse del siguiente utilizando comas.
- **WHERE [CONDICIÓN].** Esta parte de la sentencia es opcional, si se indica, la actualización de los datos sólo afectará a los registros que cumplen la condición.

Por el contrario, **si no se indica la cláusula WHERE, los cambios afectarán a todos los registros de la tabla.**

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

3.- Actualización de registros. La sentencia UPDATE.

3.1.- Sentencia Básica. Ejemplo (I).

```
DROP TABLE EMPLEADOS;

CREATE TABLE DEPARTAMENTOS (
    DEPARTAMENTOID NUMBER(3) PRIMARY KEY,
    NOMBRE VARCHAR2(20)
);

CREATE TABLE EMPLEADOS (
    EMPLEADOID NUMBER(5) PRIMARY KEY,
    NOMBRE VARCHAR2(20),
    SALARIO NUMBER(8,2),
    DEPARTAMENTOID NUMBER(3),
    CONSTRAINT EMPLEADOS_DEPARTAMENTOID_FK FOREIGN KEY (DEPARTAMENTOID)
        REFERENCES DEPARTAMENTOS(DEPARTAMENTOID)
);

INSERT INTO DEPARTAMENTOS VALUES (1, 'CONTABILIDAD');
INSERT INTO DEPARTAMENTOS VALUES (2, 'COMERCIAL');
INSERT INTO DEPARTAMENTOS VALUES (3, 'INFORMATICA');
INSERT INTO EMPLEADOS VALUES (1, 'LOPEZ', 1200, 2);
INSERT INTO EMPLEADOS VALUES (2, 'JIMENEZ', 1400, 2);
INSERT INTO EMPLEADOS VALUES (3, 'SERRANO', 1600, 1);
INSERT INTO EMPLEADOS VALUES (4, 'TORRES', 1350, 3);
INSERT INTO EMPLEADOS VALUES (5, 'LOPEZ', 1200, 2);
INSERT INTO EMPLEADOS VALUES (6, 'MENENDEZ', 1750, 1);
INSERT INTO EMPLEADOS VALUES (7, 'ALVAREZ', 1850, 1);

COMMIT;
```

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

3.- Actualización de registros. La sentencia UPDATE.

3.1.- Sentencia Básica. Ejemplo (II).

```
UPDATE EMPLEADOS  
SET SALARIO = SALARIO + 200  
WHERE SALARIO < 1500;
```

4 filas actualizadas.

```
SELECT * FROM EMPLEADOS;
```

EMPLEADOID	NOMBRE	SALARIO	DEPARTAMENTOID
1	LOPEZ	1400	2
2	JIMENEZ	1600	2
3	SERRANO	1600	1
4	TORRES	1550	3
5	LOPEZ	1400	2
6	MENENDEZ	1750	1
7	ALVAREZ	1850	1

7 filas seleccionadas.

```
ROLLBACK;
```

Rollback terminado.

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

3.- Actualización de registros. La sentencia UPDATE.

3.2.- Sentencia UPDATE con una consulta SQL. Sentencia SQL dentro de SET.

Sintaxis.

```
UPDATE tabla
SET campo = [consulta SQL]
WHERE [condición];
```

Ejemplo. Cambiar el valor del salario a el salario medio a los empleados que ganen menos de 1300.

```
UPDATE EMPLEADOS
SET SALARIO = (SELECT AVG(SALARIO) FROM EMPLEADOS)
WHERE SALARIO < 1300;

2 filas actualizadas.

SELECT * FROM EMPLEADOS;
```

EMPLEADOID	NOMBRE	SALARIO	DEPARTAMENTOID
1	LOPEZ	1478,57	2
2	JIMENEZ	1400	2
3	SERRANO	1600	1
4	TORRES	1350	3
5	LOPEZ	1478,57	2
6	MENENDEZ	1750	1
7	ALVAREZ	1850	1

```
7 filas seleccionadas.

ROLLBACK;

Rollback terminado.
```


UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

3.- Actualización de registros. La sentencia UPDATE.

3.2.- Sentencia UPDATE con una consulta SQL(II). Sentencia SQL dentro de WHERE.

Sintaxis.

```
UPDATE tabla
SET campo1 = valor1, campo2=valor2, ... , campoN=valorN
WHERE campo = [sentencia SQL];
```

Ejemplo. Subir 500€ a todos los empleados del departamento 'COMERCIAL'.

```
UPDATE EMPLEADOS
SET SALARIO = SALARIO + 500
WHERE DEPARTAMENTOID = (SELECT DEPARTAMENTOID FROM DEPARTAMENTOS
                        WHERE NOMBRE = 'COMERCIAL');
```

3 filas actualizadas.

```
SELECT * FROM EMPLEADOS;
```

EMPLEADOID	NOMBRE	SALARIO	DEPARTAMENTOID
1	LOPEZ	1700	2
2	JIMENEZ	1900	2
3	SERRANO	1600	1
4	TORRES	1350	3
5	LOPEZ	1700	2
6	MENENDEZ	1750	1
7	ALVAREZ	1850	1

7 filas seleccionadas.

```
ROLLBACK;
```

Rollback terminado.

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

4.- Borrado de registros. La sentencia DELETE.

4.1.- Sentencia Básica. Sentencia y características.

Sintaxis.

```
DELETE FROM tabla [ WHERE condición ];
```

- Se eliminan todos los registros que cumplen esa condición.
- **La tabla seguirá existiendo con la estructura que tenía hasta el momento.**
- **Si no se indica una condición, se borrará todo el contenido de la tabla.**

```
DELETE FROM EMPLEADOS WHERE DEPARTAMENTOID = 2;
```

3 filas eliminado

```
SELECT * FROM EMPLEADOS;
```

EMPLEADOID	NOMBRE	SALARIO	DEPARTAMENTOID
3	SERRANO	1600	1
4	TORRES	1350	3
6	MENENDEZ	1750	1
7	ALVAREZ	1850	1

```
ROLLBACK;
```

Rollback terminado.

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

4.- Borrado de registros. La sentencia DELETE.

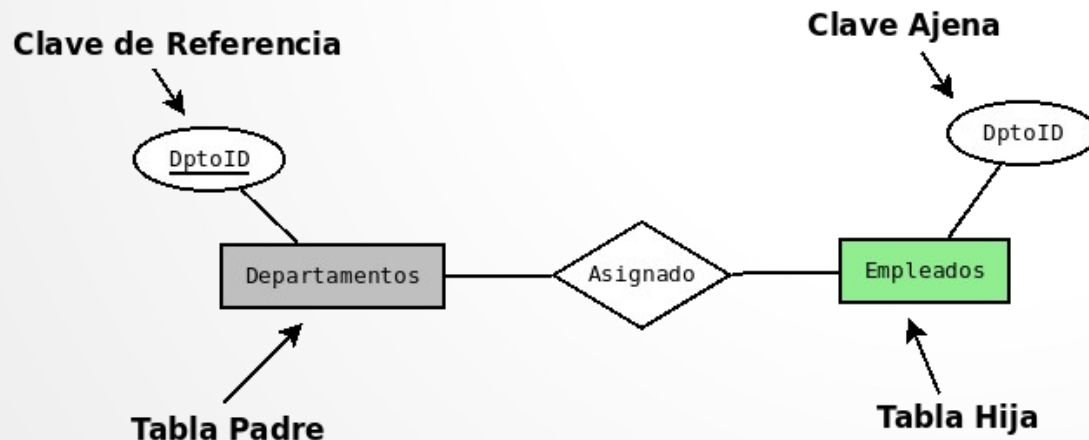
4.2.- Integridad Referencial. Definición de Borrado en Cascada.

Cuando se habla de integridad referencial se utilizan los siguientes términos:

- **Clave ajena.** Es el campo o conjunto de campos que definen de la restricción que deben hacer referencia a una clave de referencia.
- **Clave de referencia.** Clave única o primaria de la tabla a la que se hace referencia desde una clave ajena.

- **Tabla hija o dependiente.** Tabla que incluye la clave ajena, y que, por tanto, depende de los valores existentes en la clave de referencia.
- **Tabla padre o de referencia.** Corresponde a la tabla que es referenciada por la clave ajena en la tabla hija. Esta tabla determina las inserciones o actualizaciones que son permitidas en la tabla hija, en función de dicha clave.

En este caso, puede haber problemas para eliminar un departamento que tenga empleados.



UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

4.- Borrado de registros. La sentencia DELETE.

4.2.- Integridad Referencial. Opciones de Borrado en Cascada.

Cuando se hace el borrado de registros en una tabla de referencia, se puede configurar la clave ajena de diversas maneras para que se conserve la integridad referencial:

- **No Permitir Supresión (opción por defecto).** En caso de que se intente borrar en la tabla de referencia un registro que está siendo referenciado desde otra tabla, **se produce un error en la operación de borrado impidiendo dicha acción.**
- **Supresión en Cascada.** Al suprimir registros de la tabla de referencia, los registros de la tabla hija que hacían referencia a dichos registros, también son borrados. En nuestro ejemplo, al borrar un departamento, se eliminan todos sus empleados.
- **Definir Nulo en Suprimir.** Los valores de la clave ajena que hacían referencia a los registros que hayan sido borrados de la tabla de referencia, son cambiados al valor NULL.

UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

5.- Transacciones.

5.1.- Validar cambios. Sentencia COMMIT.

- El resultado de un conjunto de sentencias DML no es permanente hasta que se hace la operación COMMIT.

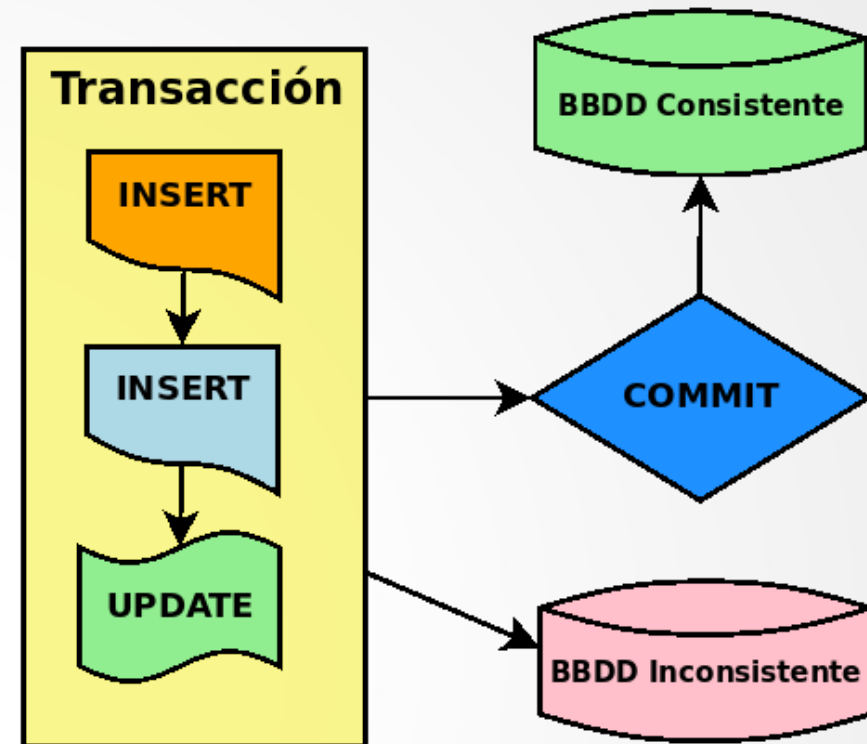
Si no hemos hecho ningún COMMIT y el SGBD cae por cualquier motivo nuestras tablas estarán en la situación de partida.

- ¿Cuándo se produce un COMMIT?
 - Al utilizar la sentencia COMMIT.
 - Al ejecutar una sentencia DDL.
 - Al cerrar adecuadamente las aplicaciones de gestión de las bases de datos Oracle.
- **Parámetro AUTOCOMMIT.** Validar automáticamente las transacciones sin tener que indicarlo de forma explícita.

SQL> SHOW AUTOCOMMIT;

El valor por omisión es OFF. En algunos SGBD tales como MySQL o MariaDB la opción AUTOCOMMIT está activada por defecto.

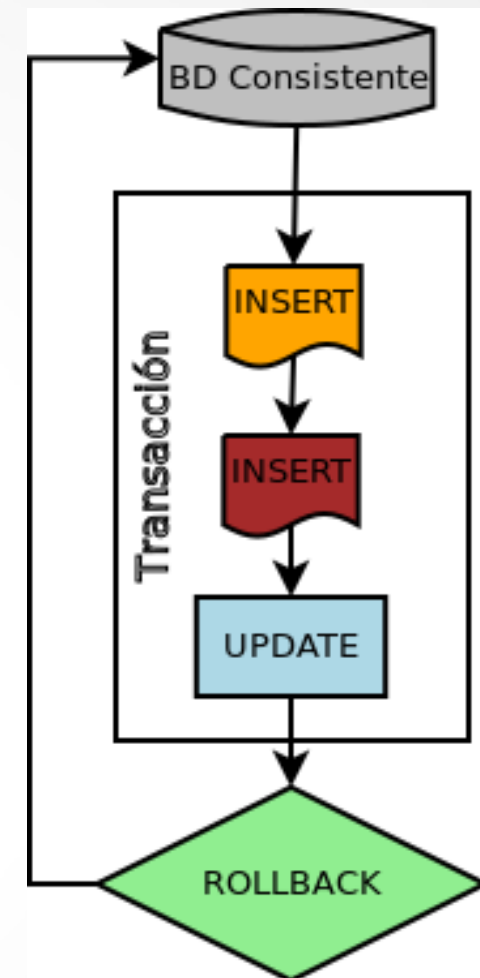
- Se recomienda que se finalice las transacciones en las aplicaciones usando las sentencias COMMIT. o ROLLBACK.



Formato:

SQL> ROLLBACK;

- Aborta la transacción volviendo a la situación de las tablas de la base de datos desde el último COMMIT o el arranque de la Base de Datos.
- Si se produce un fallo del sistema, Oracle hace un ROLLBACK automático sobre cualquier trabajo no validado.



UT 04 - El Lenguaje DML. Manipulación de Datos de las Tablas.

5.- Transacciones.

5.3.- Puntos de Restauración. Sentencia SAVEPOINT.

- **Un punto de restauración (SAVEPOINT).** Marcador intermedio declarado por el usuario en el contexto de una transacción. Los puntos de restauración dividen una transacción grande en pequeñas partes.

- **Formato del Comando:**

SAVEPOINT puntoRestauración;

donde puntoRestauración es una cadena con el nombre identificativo con el que se le desea conocer.

- La **restauración de los cambios hasta ese punto** se hará con un comando con el siguiente formato:

ROLLBACK TO SAVEPOINT puntoRestauración;

